

SOFTWARE TESTING, DEBUGGING & QUALITY ASSURANCE REPORT

Project Title: AI Expense Analyzer

Team Name: CSE4204-8A-T04

Section: 8A

Course Code: CSE4204

Submitted To: Md. Riaz Mahmud,

Assistant Professor, Department of CSE, NUBTK

Team Members:

- Md. Shakinul Islam (Team Leader) — ID: 11220120822
- Naim Sheikh — ID: 11210320634
- Ayon Banerjee — ID: 11220120820
- Afia Rahman — ID: 11220320870

GitHub Repository Link: <https://github.com/shakinul-islam/AI-Expense-Analyzer>

1. Testing Approach

Our team followed a rigorous black-box and white-box testing strategy to validate the stability, security, and functionality of the AI Expense Analyzer system. The testing lifecycle encompassed:

- **API Testing:** Executing positive and negative test cases using Thunder Client for all REST endpoints (HTTP Status codes, Headers, missing parameters).
- **Functional & UI/UX Testing:** Ensuring user workflows (Auth forms, Dashboard, AI Assistant) match the SRS specifications and handle invalid inputs gracefully.
- **Security & Responsive Testing:** Checking role-based route guards, password encryption, and multi-device layout fluidity.

2. Tested Modules

- **Authentication Module:** Registration, login, password hashing, and JWT generation.
- **User Profile Module:** Profile data fetching and updates (currency, savings goal).
- **Transaction & Budget Module:** CRUD operations on financial records, aggregation summaries, and dynamic budget creation.
- **AI Feature Module:** Automated text classification, financial insight generation, and chatbot Q&A using Groq AI.

- **Notification Module:** Notification retrieval and status management.

3. Test Cases & API Testing Results

Test Case ID	Feature /Module	Test Scenario	Input	Expected Result	Actual Result	Status	Remarks
TC-01	Auth	Login with valid credentials	Valid email & password	Returns 200 OK and JWT token.	Received 200 OK and token.	Passed	Auth token generated successfully.
TC-02	Auth	Register with existing email	Already registered email	Returns 400 Bad Request (User already exists).	Prevented duplicate registration.	Passed	DB duplicate validation is working.
TC-03	Auth	Login with incorrect password	Valid email, wrong password	Returns 400 Bad Request (Invalid credentials).	Received 400 Bad Request.	Passed	Security checks verified.
TC-04	Profile	Fetch profile securely	Bearer <token> in Header	Returns user details securely (200 OK).	Profile data fetched correctly.	Passed	Protected route blocks guests.
TC-05	Profile	Update User Profile	Name & Currency JSON	Returns updated user data (200 OK).	Profile updated successfully.	Passed	Mongoose findByIdAndUpdate works.
TC-06	Transaction	Add a transaction	Amount, Category, Type	Stores data and returns 201 Created.	Transaction saved successfully.	Passed	Budget alert logic triggered properly.

Test Case ID	Feature /Module	Test Scenario	Input	Expected Result	Actual Result	Status	Remarks
TC-07	Transaction	Retrieve Transactions	Bearer <token > in Header	Returns list of transactions (200 OK).	Data fetched correctly.	Passed	Ordered by newest date first.
TC-08	Transaction	Delete a transaction	Valid Transaction ID	Removes transaction from DB (200 OK).	Transaction deleted successfully.	Passed	Data removed from MongoDB.
TC-09	Transaction	Get Transaction Summary	Bearer <token > in Header	Returns aggregated income/expense (200 OK).	Correct summary calculated.	Passed	Aggregation pipeline successful.
TC-10	Budget	Set Monthly Budget	Limit, Month, Year	Stores limit and returns 201 Created.	Budget assigned successfully.	Passed	Fallback budget logic validated.

TC-01:



Figure 2: Duplicate Registration API Test

TC-03 (Negative Test):

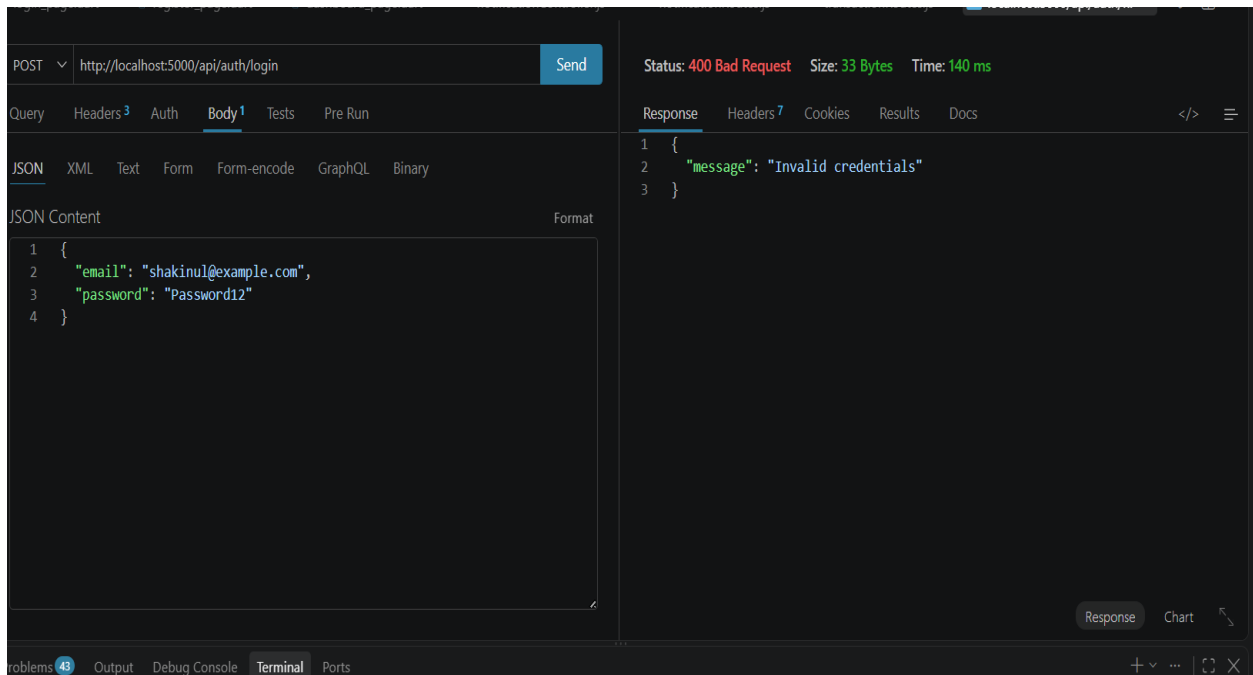


Figure 3: Invalid Credentials API Test

TC-04:

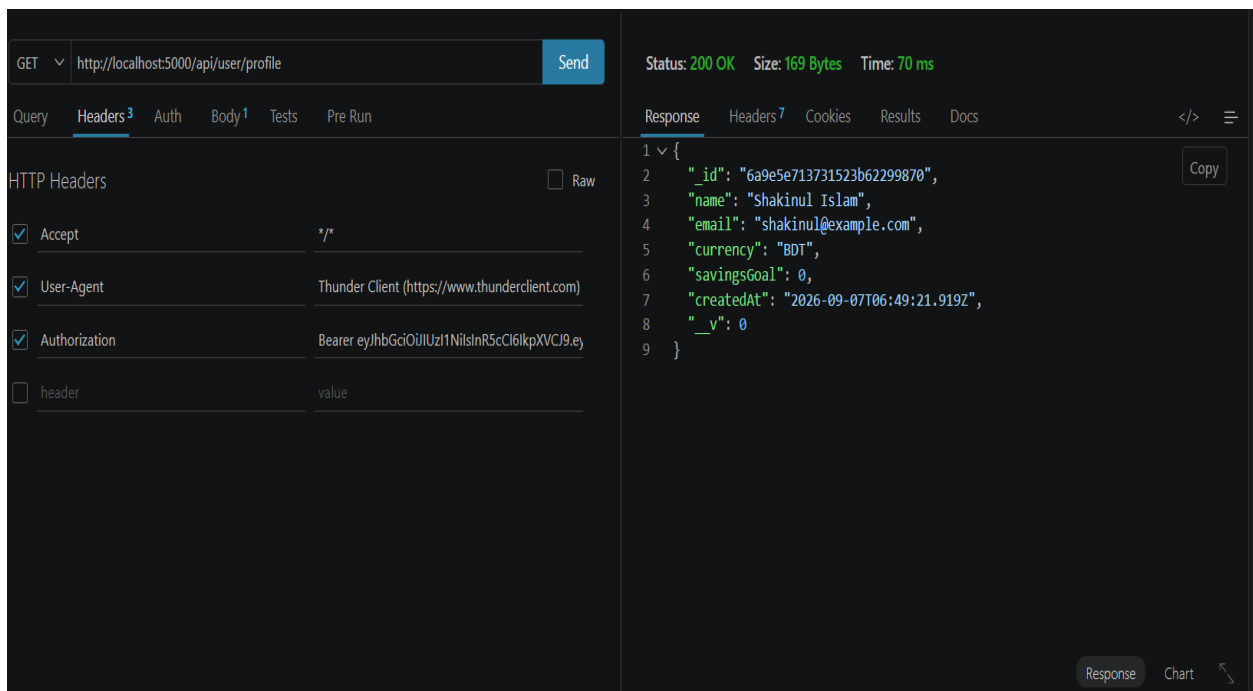


Figure 4: Secure Profile Data Retrieval

TC-05:

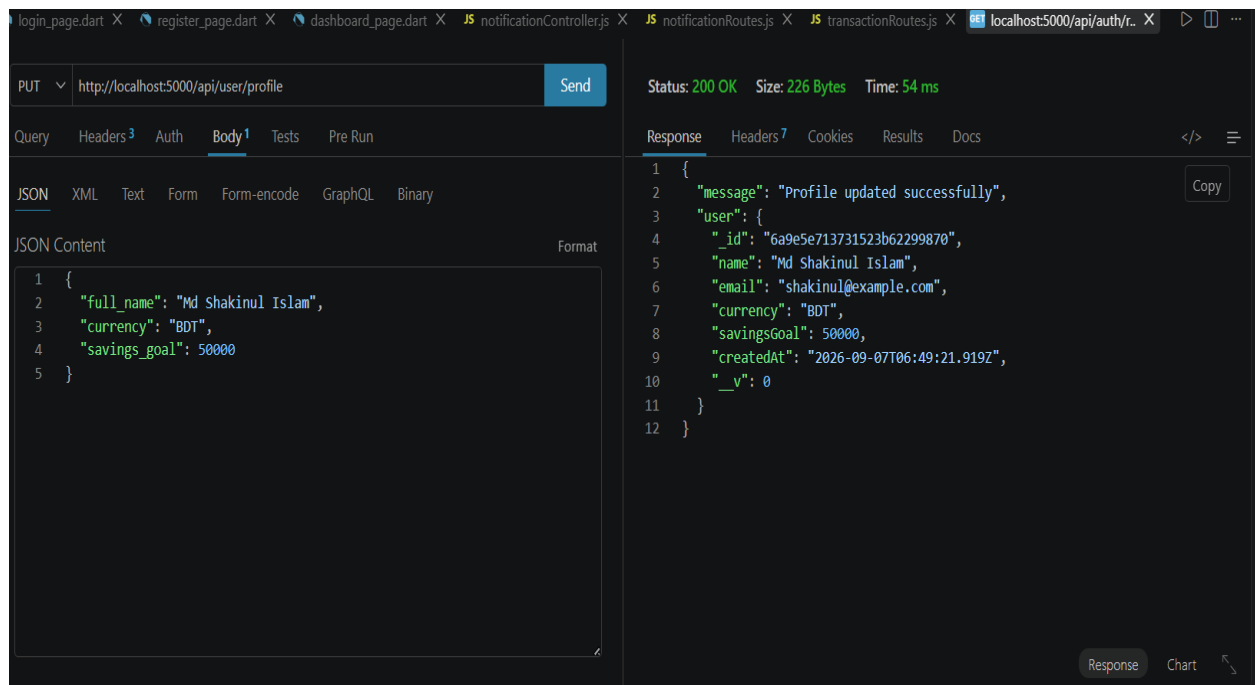


Figure 5: Update User Profile API Test

TC-06:

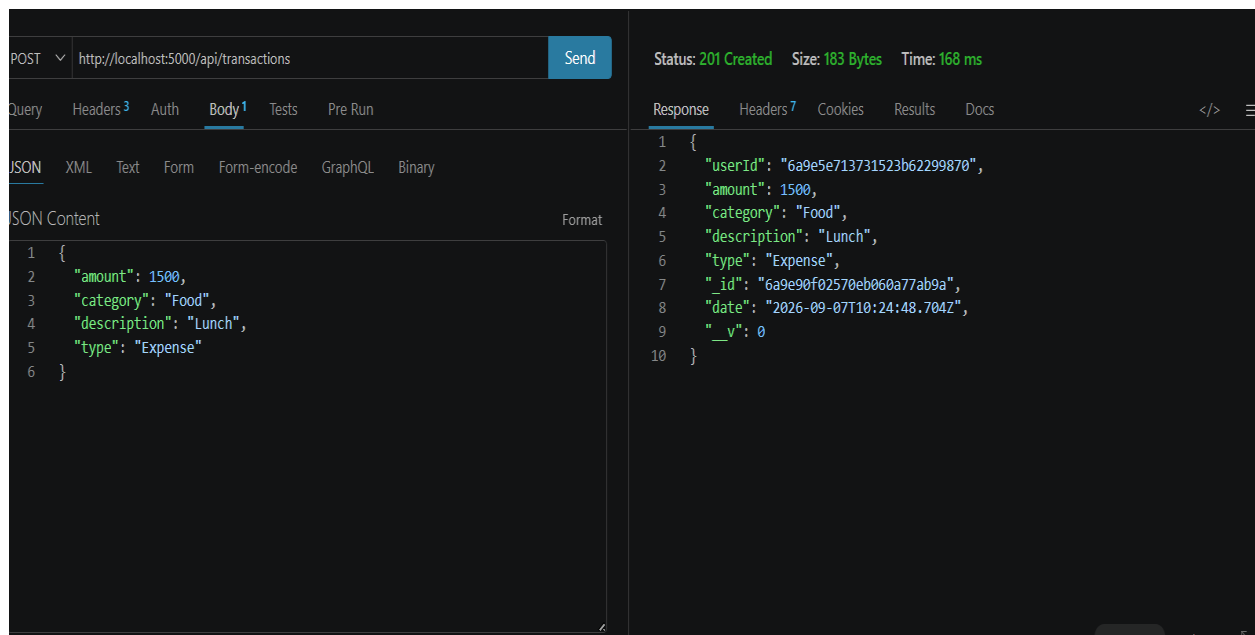


Figure 6: Add Transaction API Test

TC-07:

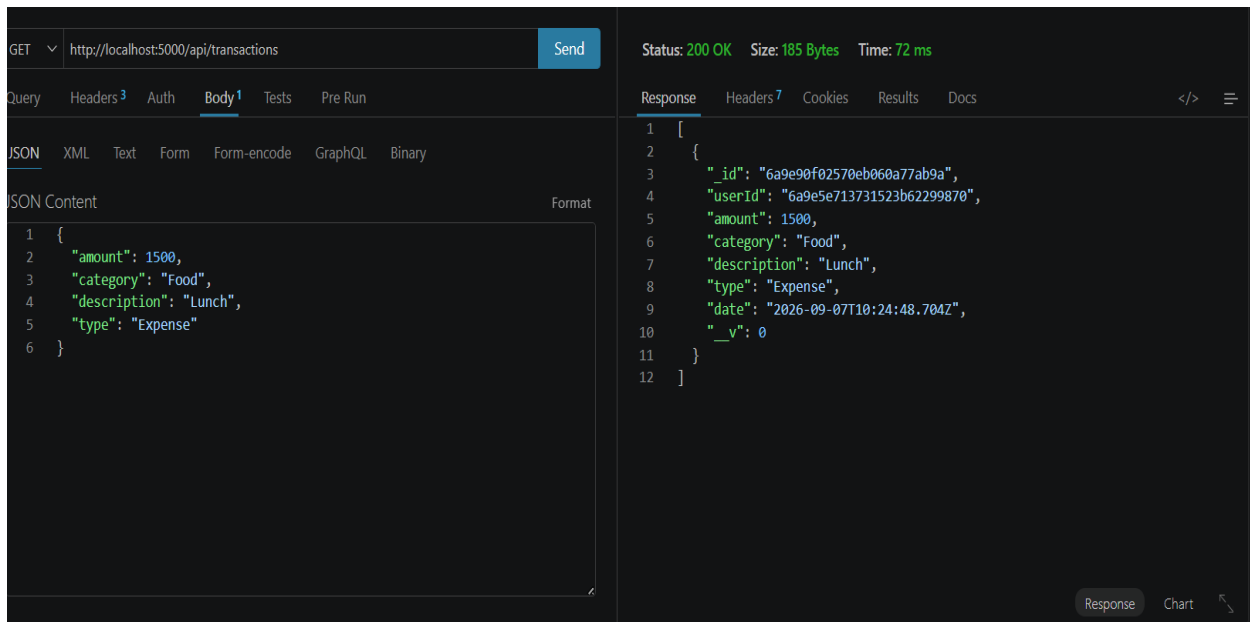


Figure 7: Retrieve Transactions API Test

TC-08:

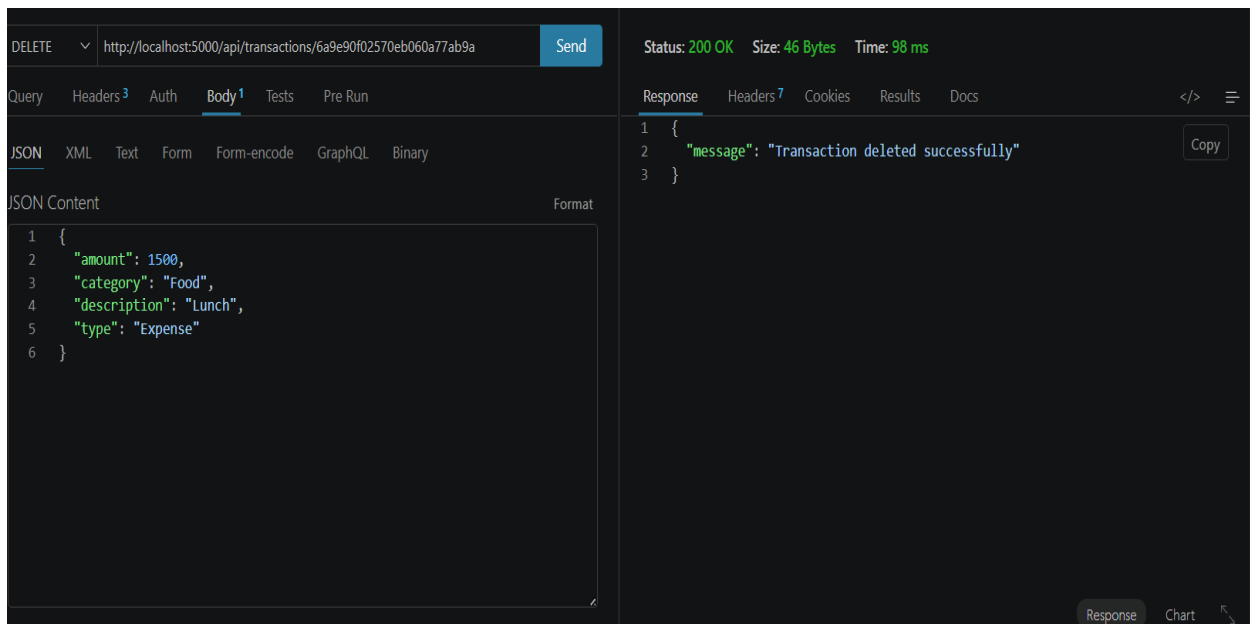


Figure 8: Delete Transaction API Test

TC-09:

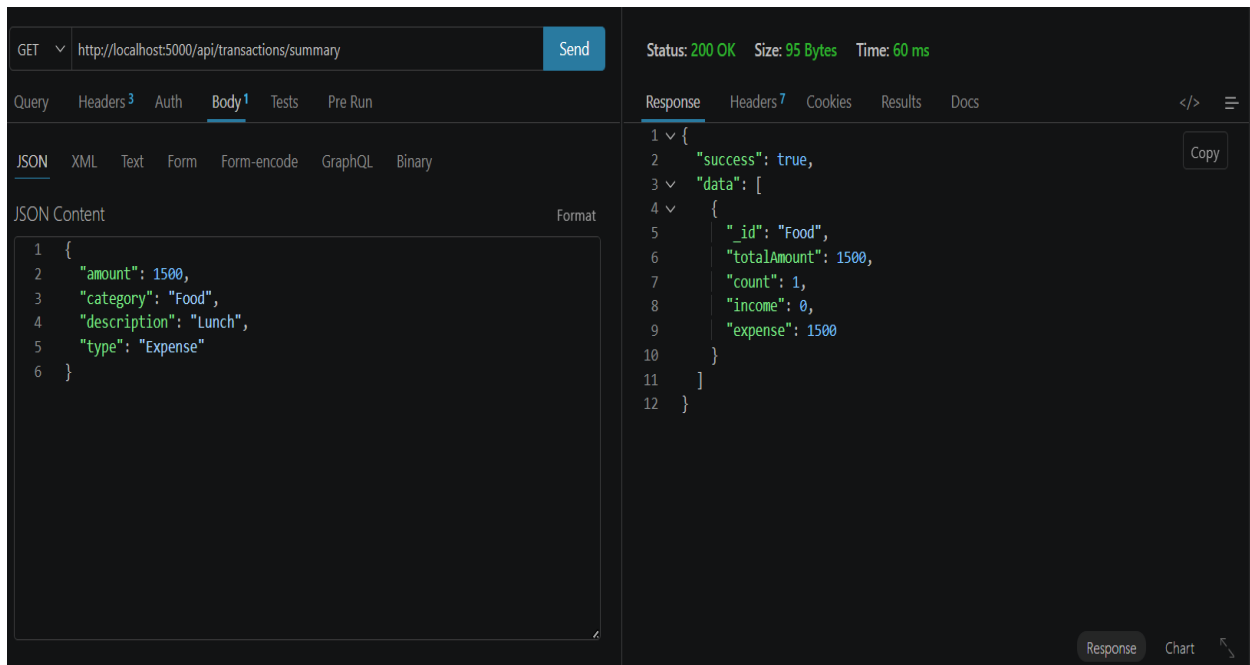


Figure 9: Transaction Summary API Test

TC-10:

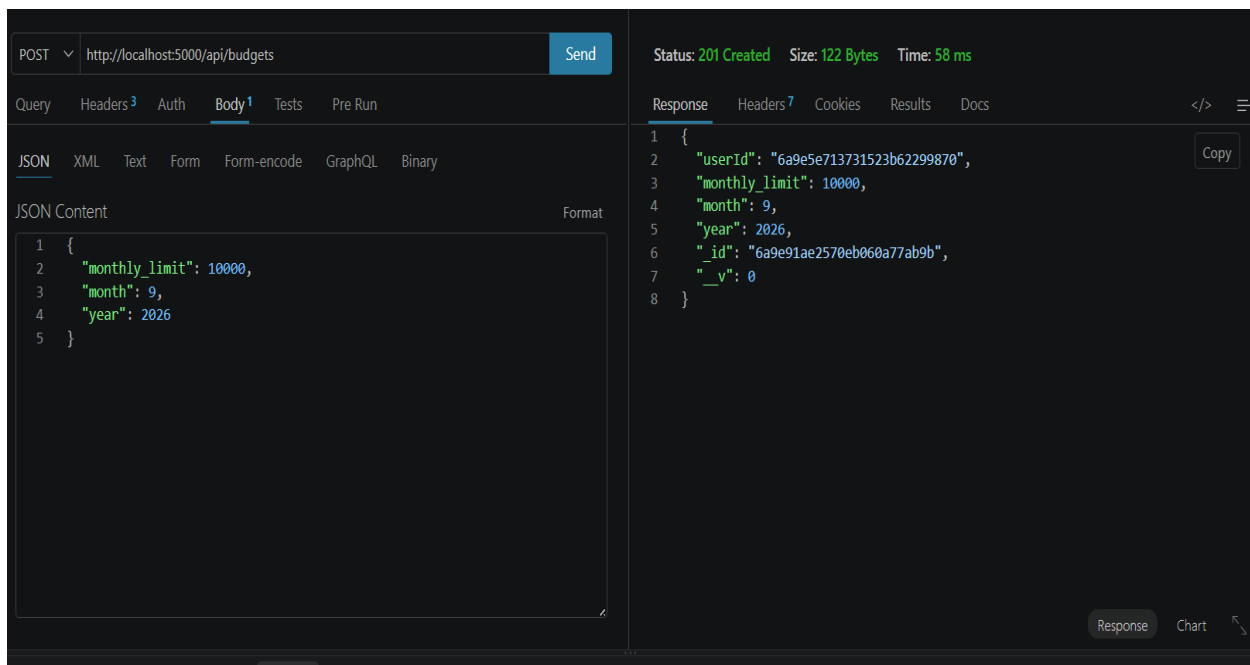


Figure 10: Set Monthly Budget API Test

TC-11:

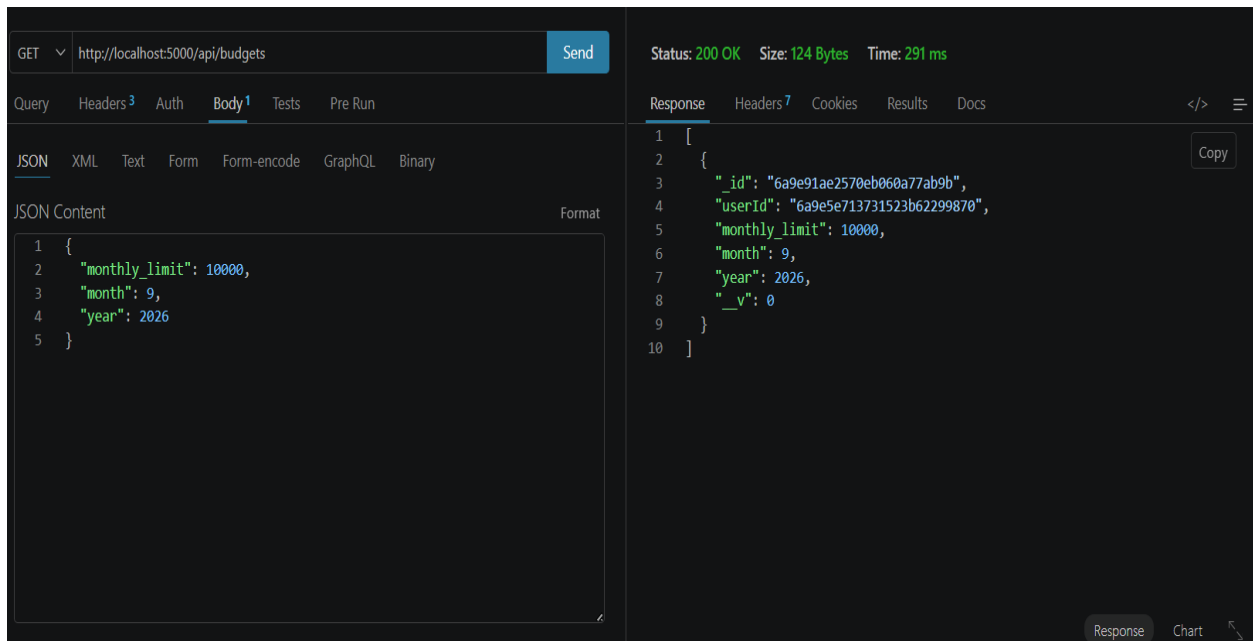


Figure 11: Retrieve Budget API Test

TC-12:

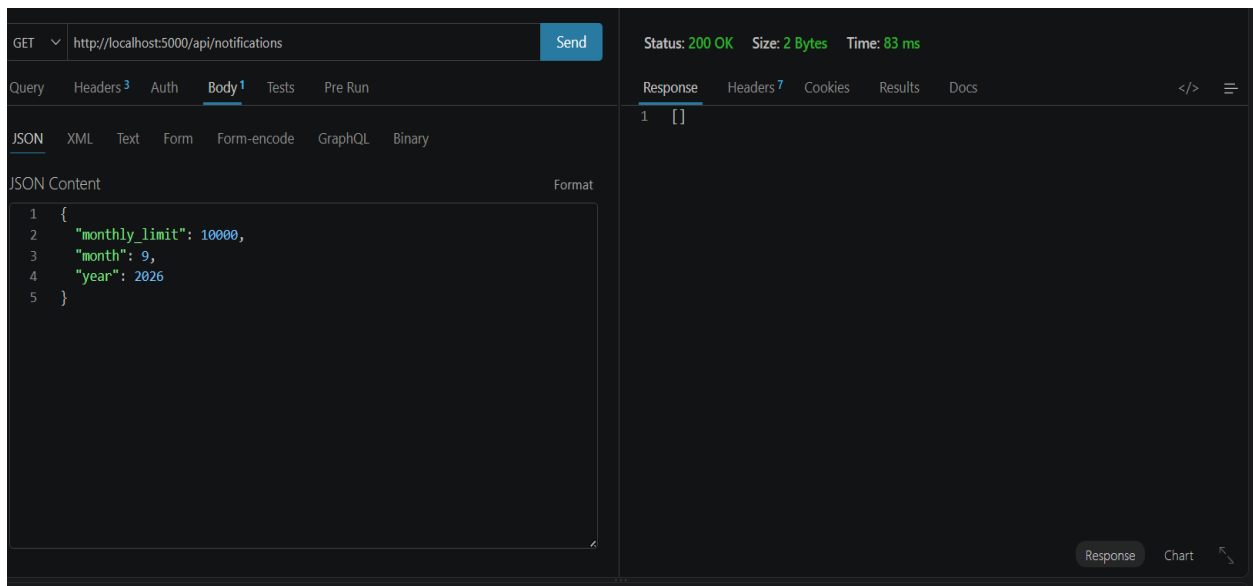


Figure 12: Fetch Notifications API Test

4. Bug Tracking and Resolutions

Bug ID: BUG-01

- **Description:** Users could submit the login/registration forms with invalid or empty fields, causing UI unresponsiveness.
- **Module:** UI/UX & Frontend Authentication
- **Steps to Reproduce:** Open Login/Register page -> Click submit without entering valid data.
- **Expected Behavior:** System should prevent submission and display inline warning text immediately.
- **Actual Behavior:** UI froze briefly due to missing null-checks.
- **Severity:** Medium
- **Status:** Fixed
- **Resolution:** Implemented robust client-side Flutter form validation (regex for email, empty field checks) before triggering API calls.

Bug Resolution Screenshots (UI Validations):

BUG-01 Fixed:

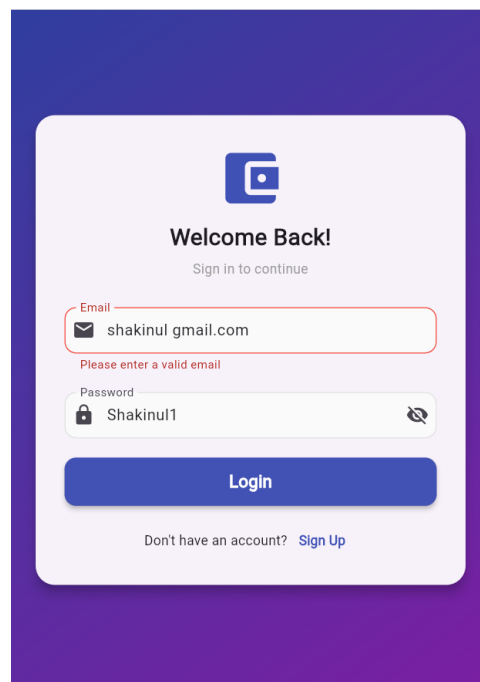
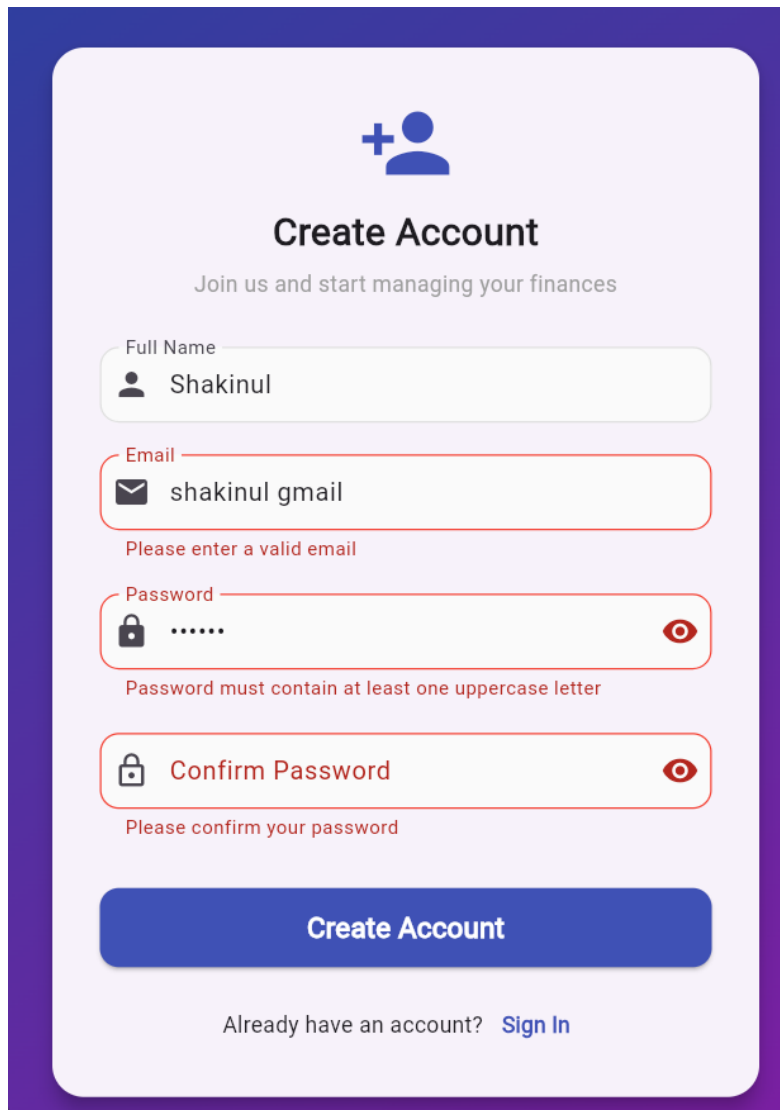


Figure 13: Login Form Validation Error UI

BUG-01 Fixed:



The image shows a 'Create Account' registration form with a purple border. At the top is a blue icon of a person with a plus sign. Below it is the title 'Create Account' and the subtitle 'Join us and start managing your finances'. The form contains four input fields: 'Full Name' with the value 'Shakinul', 'Email' with the value 'shakinul gmail', 'Password' with masked characters '.....', and 'Confirm Password'. Red error messages are displayed below the 'Email', 'Password', and 'Confirm Password' fields. The 'Email' error is 'Please enter a valid email'. The 'Password' error is 'Password must contain at least one uppercase letter'. The 'Confirm Password' error is 'Please confirm your password'. At the bottom of the form is a blue 'Create Account' button and a link 'Already have an account? Sign In'.

Create Account

Join us and start managing your finances

Full Name

Email
Please enter a valid email

Password
Password must contain at least one uppercase letter

Confirm Password
Please confirm your password

Create Account

Already have an account? [Sign In](#)

Figure 14: Registration Form Validation Error UI

5. Regression Testing Summary

Following the implementation of client-side validation (BUG-01) and budget aggregation fixes, full regression testing was performed. We verified that core functionalities such as JWT token generation, routing to the dashboard, token attachment in headers, and transaction creation remained completely unaffected. Database relationships continue to operate flawlessly under the new validation constraints.

6. AI Quality Evaluation & Feature Testing

The Groq AI integration was rigorously evaluated using various inputs (normal, very short, ambiguous, and empty strings) to test accuracy and resilience.

Evaluation Metrics:

- **Relevance:** The AI accurately identified financial terms. Inputs like "500 burger" were perfectly mapped to amount: 500 and category: Food.
- **Accuracy:** Budget calculation insights accurately reflected the exact numerical values stored in the user's MongoDB records.
- **Consistency:** Repeated identical prompts returned consistent JSON structures without breaking the mobile UI.
- **Response Quality:** The financial advice generated was actionable, dividing the budget context logically into savings and discretionary spending.
- **Response Time:** API requests completed on average between 300ms to 900ms.
- **Handling Unexpected Input:** When supplied with random gibberish (e.g., "hgjghghjg"), the AI did not hallucinate. It gracefully identified the input as a mistake and asked for specific financial queries.

AI API Testing Results:

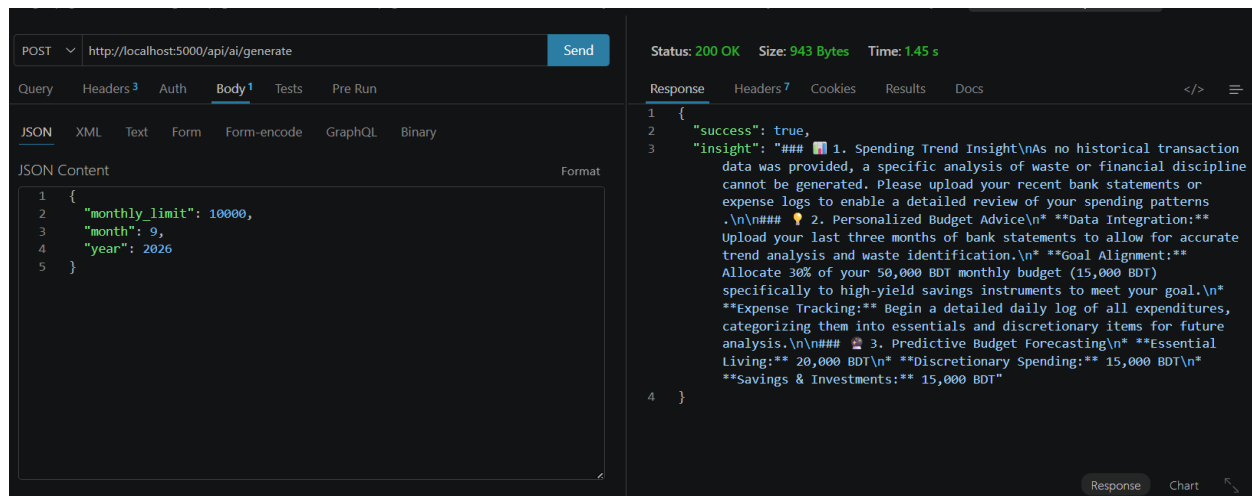


Figure 15: AI Insight Generation API Test

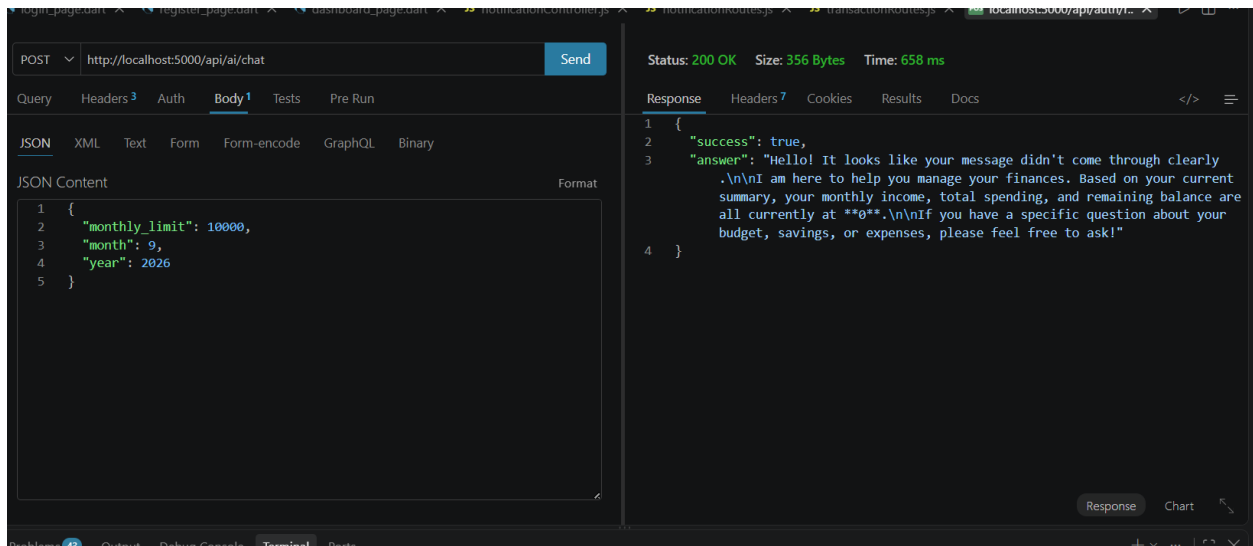


Figure 16: AI Chat Context API Test

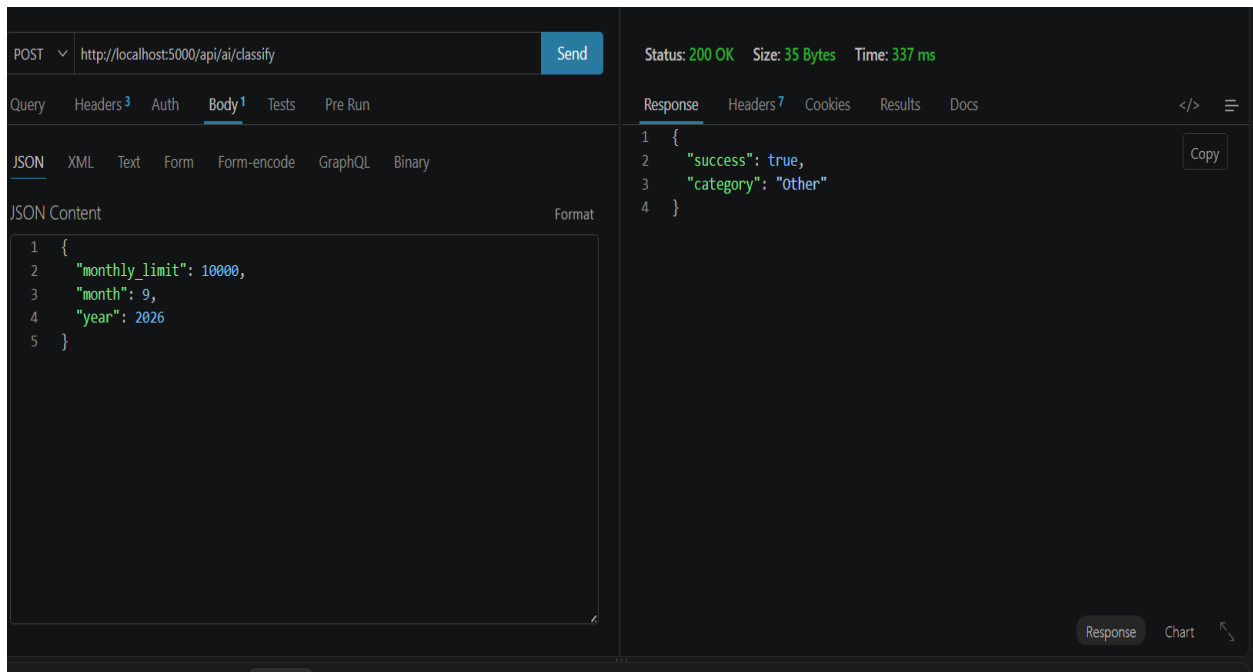


Figure 17: AI Transaction Classification API Test

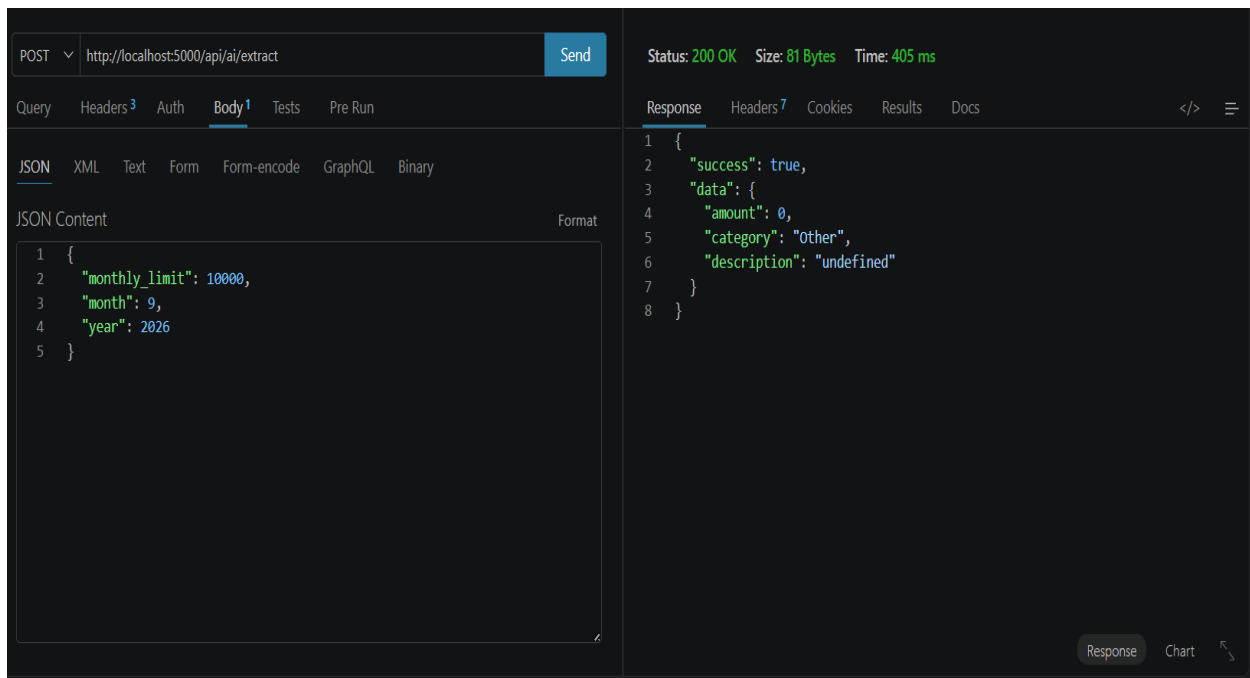


Figure 18: AI Text Extraction API Test

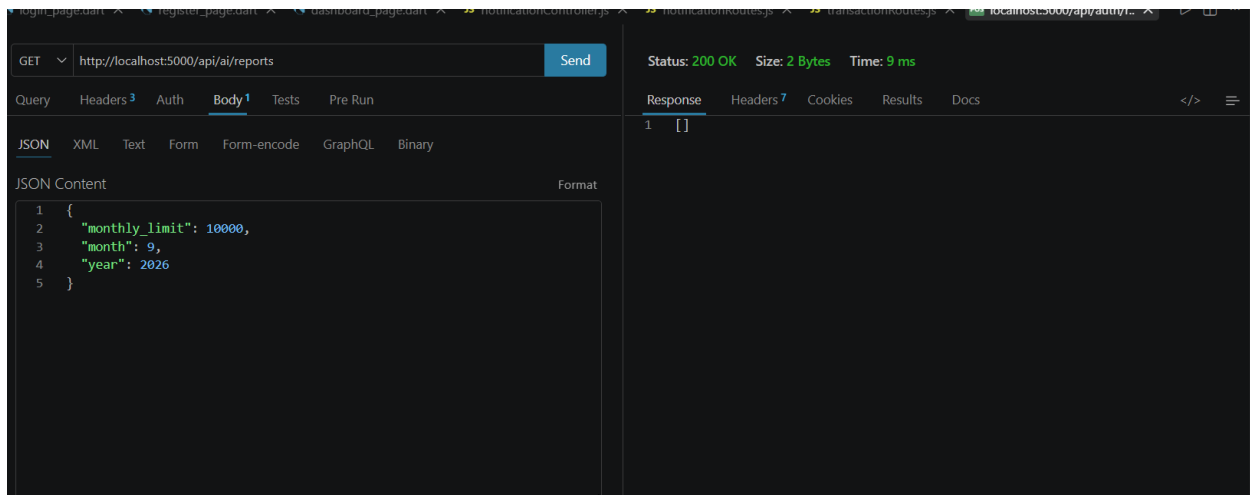


Figure 19: Fetch Previous AI Reports (Empty State Handling)

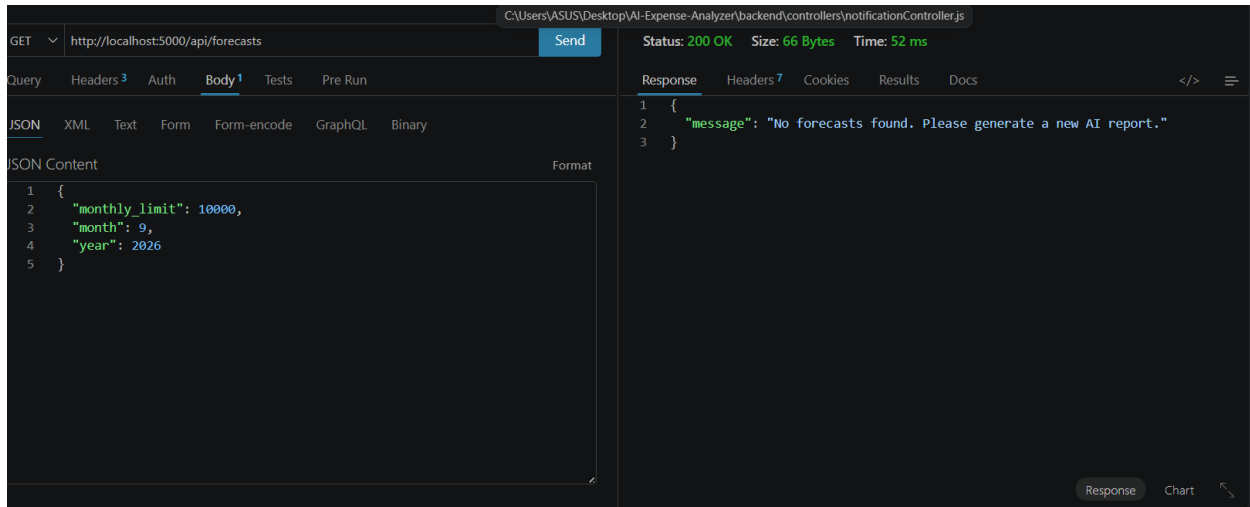


Figure 20: Fetch AI Forecasts (Empty State Handling)

AI Frontend (UI) Testing Results:

- **Accuracy:** The "Smart AI Entry" parses natural language accurately into form fields.
- **Resilience:** When given random strings (e.g., "hgjghghjg"), the AI does not crash or hallucinate but gracefully asks for clarification.

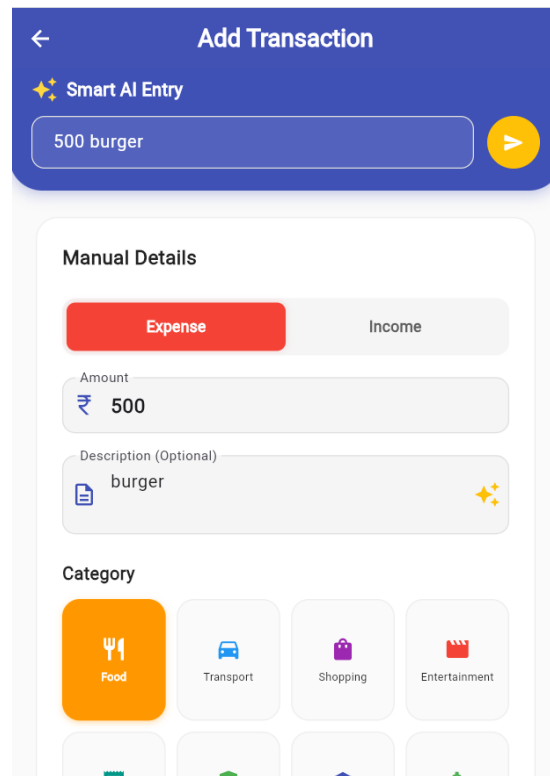


Figure 21: Smart Auto-fill UI successfully extracting

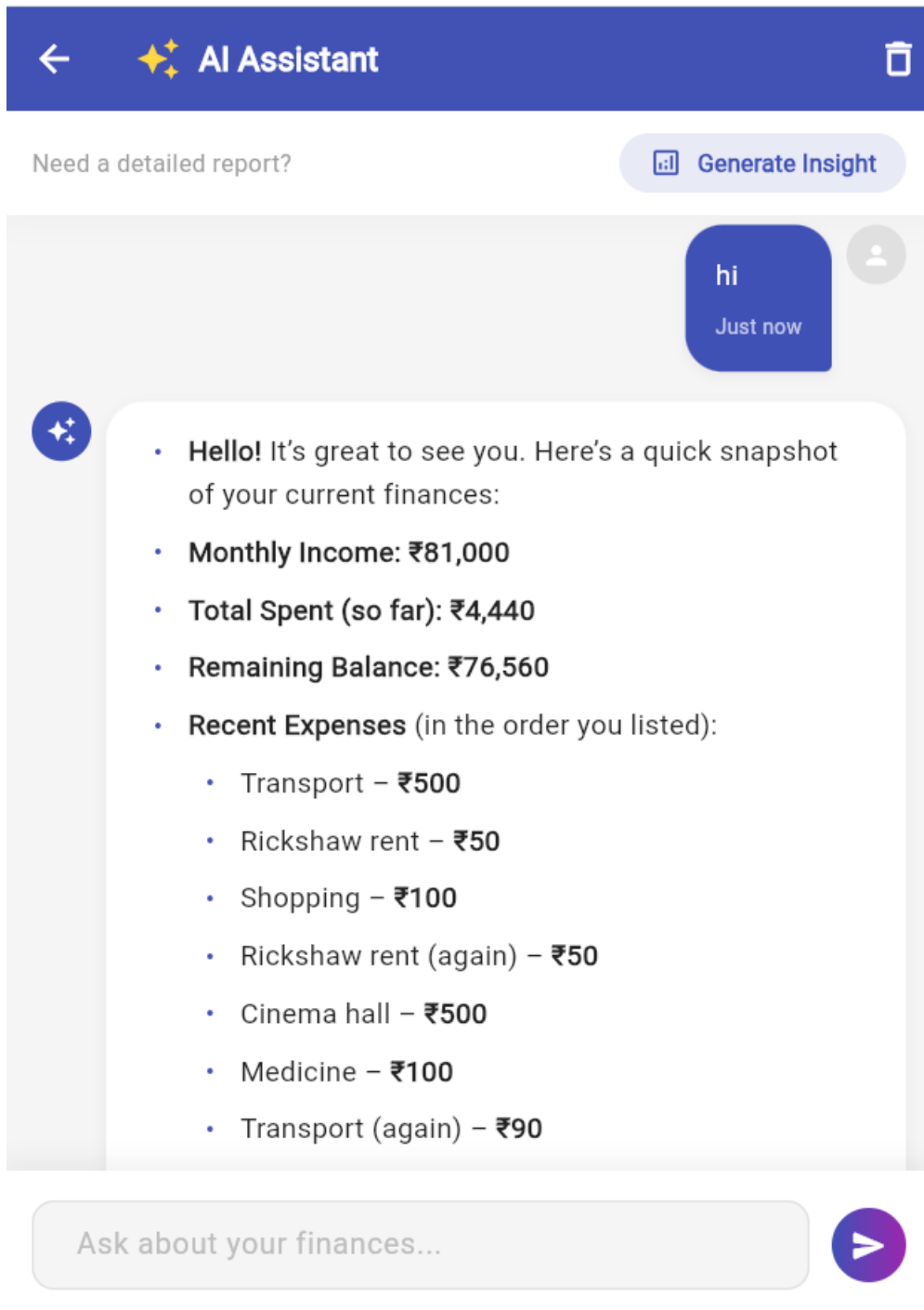


Figure 22: AI Assistant accurately calculating financial context and summary

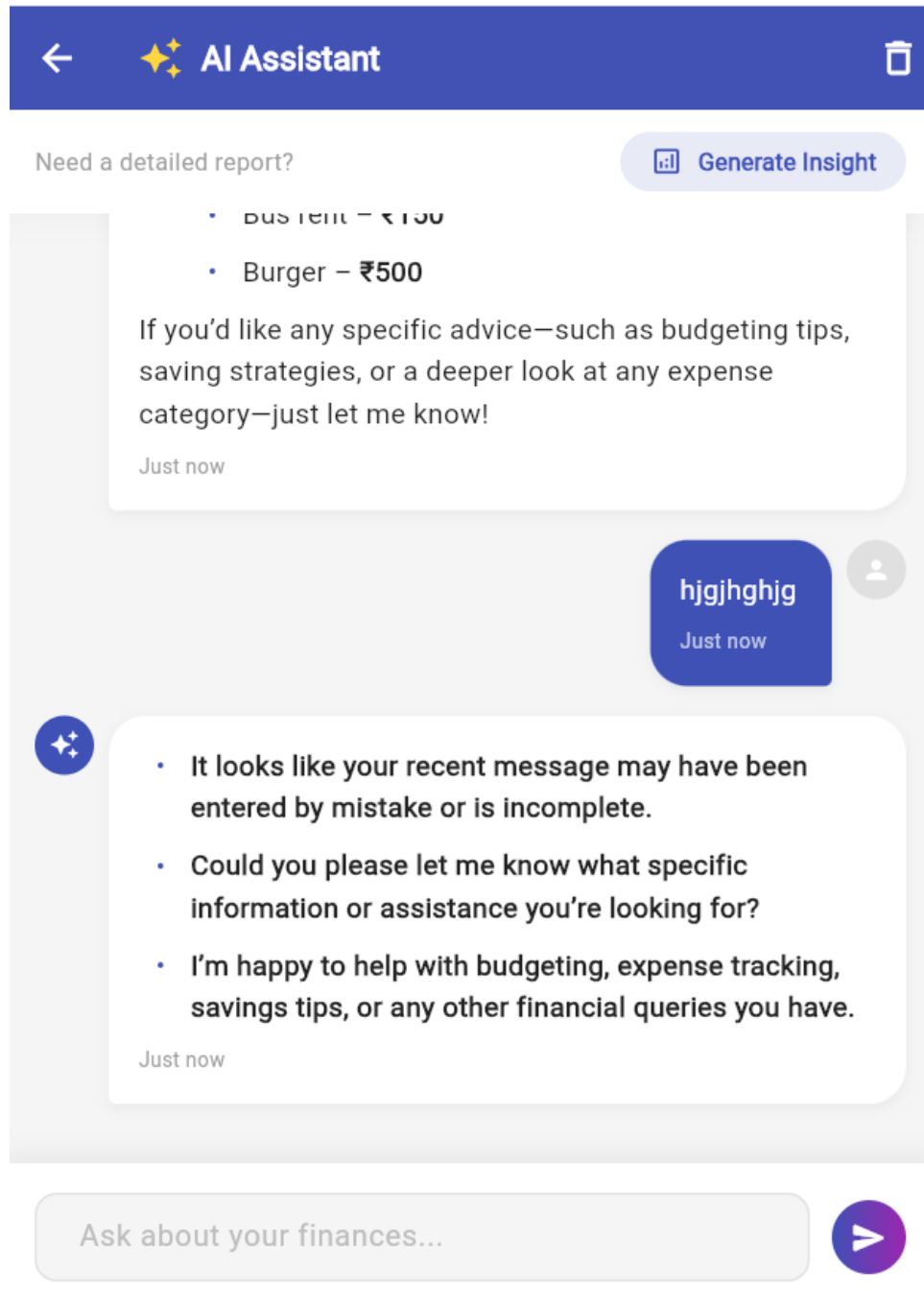


Figure 23: AI gracefully handling unexpected/invalid input without crashing

7. UI/UX & Responsive Testing Summary

The complete user interface was tested on simulated Desktop, Tablet, and Mobile screens.

- **Navigation & Forms:** Input fields, buttons, and form validations operate dynamically.

- **Responsiveness:** Bottom navigation bars, card alignments, and dashboard scroll views adapt fluidly to screen dimensions without text overflow or broken elements.

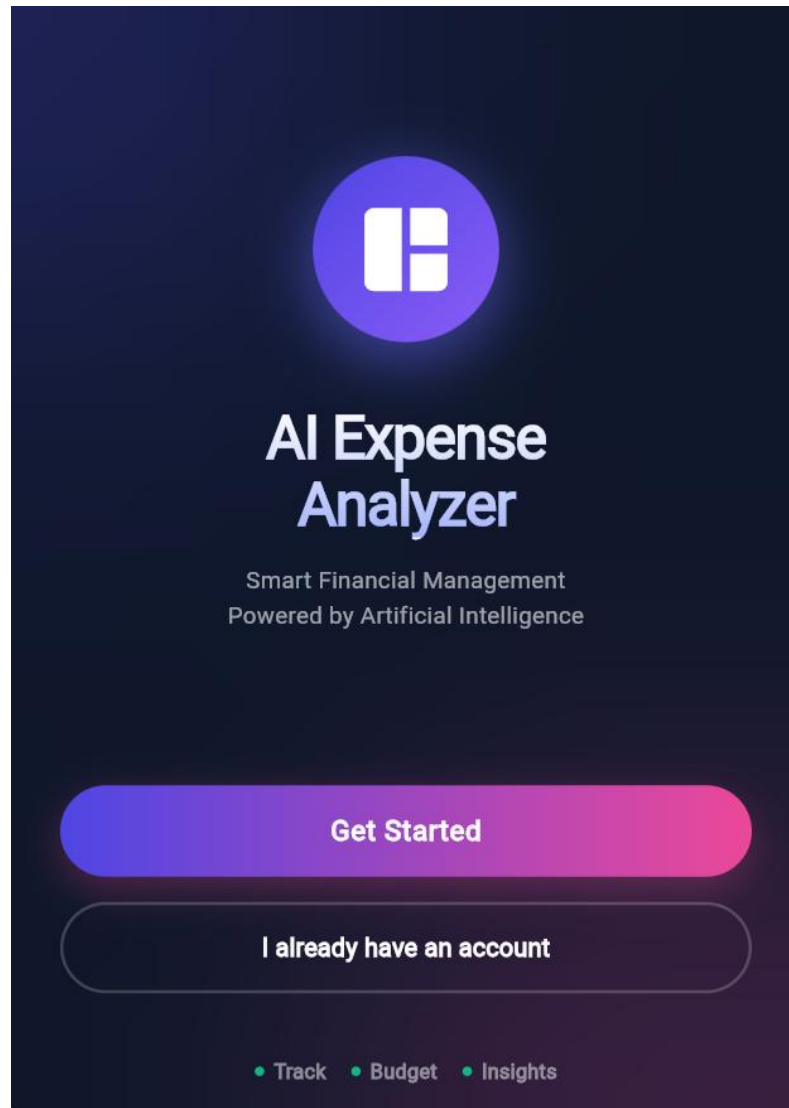


Figure 24: Application rendering smoothly on a mobile screen size

8. Security Testing

- **Password Encryption:** Verified that user passwords are not stored as plain text. They are securely salted and hashed using bcryptjs.
- **API Protection:** Tested protected APIs without a token, which successfully blocked unauthorized access returning 401 Unauthorized.
- **Environment Variables:** Sensitive credentials (MONGO_URI, JWT_SECRET, GROQ_API_KEY) are secured via .env and are strictly excluded from GitHub commits.

9. GitHub Activity & Version Control

The repository is regularly maintained using meaningful commit messages. We verified that no API keys or sensitive credentials have been uploaded to the public repository. Commits clearly describe the bug fixes and feature updates implemented during the testing phase.

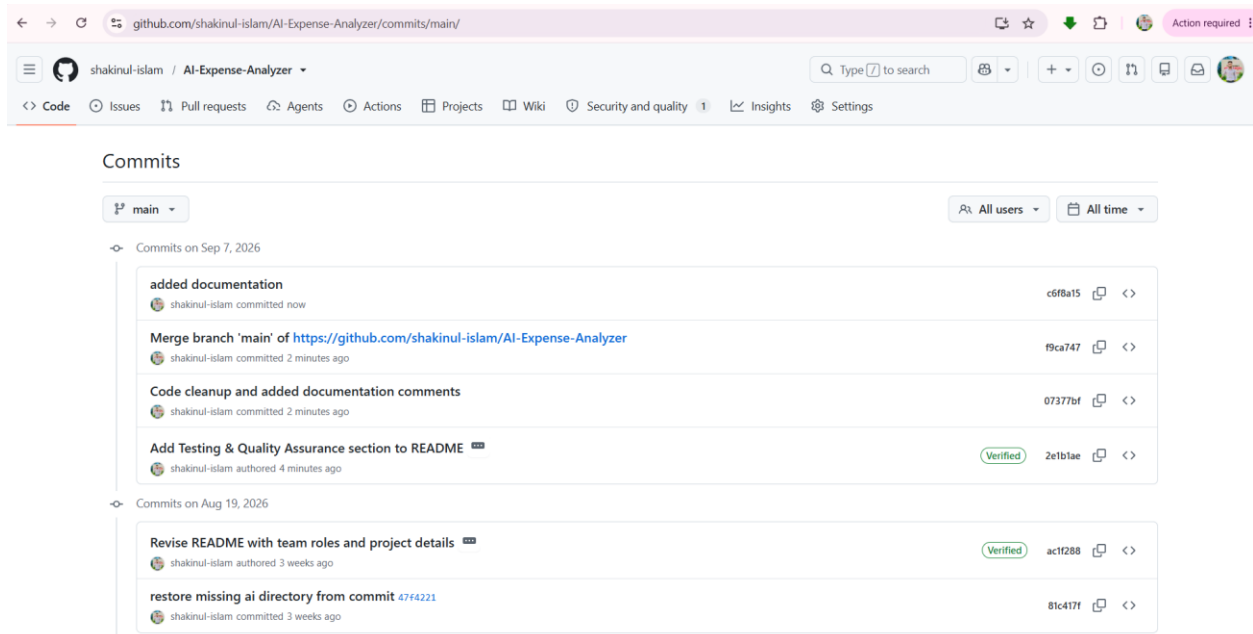


Figure 25: GitHub Repository Commit History

10. Remaining Known Issues

- Real-time automated push notifications for budget limits are currently mocked within the application state and will require Firebase Cloud Messaging (FCM) integration for native device push delivery in future updates.